

Why We Should Integrate Couchbase Lite with Android Studio

Couchbase Lite is an embedded NoSQL database for mobile devices that runs locally on the device with full CRUD and query capabilities. In this article, we discover how to integrate Couchbase Lite with Android Studio.

Android for mobile devices currently comes with an inbuilt local database—SQLite. This is an RDBMS based lightweight database that is available by default on all Android operating systems and provides CRUD

operations to efficiently power your apps. SQLite is really a great choice when the requirement is just to have a simple database for your app to manage structured data. However, when the need is to store semi-structured or unstructured data and also to handle complex queries at scale without worrying about the schema of tables, then SQLite may not suit all of the developer's requirements. A NoSQL database can be a better fit with these scaling requirements. Comparisons between an SQL and a NoSQL database have fuelled many debates, but both complement each other rather than compete with each other.

In this article, we start by discussing the general database requirements for mobile devices, followed by NoSQL's prominence in today's mobile world and, finally, look at integrating a NoSQL database called Couchbase Lite with Android Studio.

Databases for mobile devices

Deciding on a database for mobile devices requires us to consider various factors like memory constraints, the user experience, a lightweight UI, etc – parameters that are very different compared to what would be required for a desktop or Web environment. So before we jump into integrating Couchbase Lite with Android, let us first check out the various requirements for databases in the mobile environment, which are listed below.

- Unlike desktops or servers, mobile devices tend to have a lower battery life and relatively slower CPUs. Databases should hence not be performance intensive and should be able to effectively perform frequent operations like searches and updates.
- Mobile databases should have a smaller memory footprint. In certain ways, higher memory requirements would also lead

to increased CPU cycles, where the kernel tries to intensively search for available memory space in the RAM. Lower footprint demands not only lead to lower CPU cycles but also ensure other mobile apps don't get impacted.



- All the winning mobile apps are high performance with a fast loading time. Apps that freeze constantly are always on the backbench. Data consistency is another requirement when talking about these local databases on the mobile. If one were to go with a distributed database, the data may become inconsistent with respect to its remote counterpart if not taken

care of, and the device might even discover this only on connecting to the Internet.

A mobile developer with a cloud based database backend, like say Firebase, needn't worry about most of these constraints and requirements, but needs to include these factors in the equation when opting for a local database.

NoSQL's relevance in the mobile world

With the increased usage of mobile devices, a tremendous amount of data is being generated these days. This fact, clubbed with technology proliferation into new spaces like mobility, IoT and analytics, has led to a demand for mobile devices and apps to handle this high volume of data at a high speed. Besides, the nature of data (especially when it comes from IoT devices for which data exchange is in real-time) is continuous and either semi-structured or the application needs to cater or adapt to various schemas. Some of the fundamental philosophies that NoSQL brings in address these challenges in the mobile space, as discussed below.

- The very fact that NoSQL is schema-less will help developers handle the data that lacks schema or structure. Besides, this property will also let them scale to changing or evolving requirements of data. The change in schema or structure could be done with ease at any point in time, in an independent way, without affecting the existing code. All of this will directly result in the agile delivery of apps, a quick turnaround time to market as against